Code agents and code LLMs

Architecture 7: University teaching deck

What makes an agent that writes, runs, and fixes code different, and how is the model behind it trained?

Users and builders of code agents, code tools and sandboxes, context management, software engineering agents, a coding TinyAgent, and how code LLMs are trained.

Based on An Illustrated Guide to AI Agents
Maarten Grootendorst and Jay Alammar (O'Reilly, 2026)

Learning outcomes and conceptual route

Code agents and tools

Explain how a code agent extends code generation by executing code and iterating. Compare file tools, sandboxes, command-line access, computer use, code search, and SQL tools, with their security trade-offs.

Context and engineering

Describe how prompt caching, repository maps, and compaction keep long trajectories affordable. Explain SWE-bench, planning, and the agentless alternative for software engineering tasks.

Building and training

Outline the coding TinyAgent: tools, approval, and a terminal display. Describe how a code LLM is trained: code-heavy data, tool-use and test data, RLVR, and synthetic data.

Source: Ch. 10, PDF pp. 501–546

Why code is such a good target

Early and potent

Code generation was one of the first applications of LLMs once they reached the scale of GPT-3 in 2020. Coding agents are among the agents with the largest likely impact.

Knowledge work as code

An enormous range of knowledge work can be expressed as code: software engineering, research, data analysis, and visualization.

Verifiable results

Code can often be checked automatically: you run it or test it and get a clear signal of whether it worked. That makes coding more tractable than open-ended tasks with no objective check.

Who uses and builds code agents

Three kinds of users

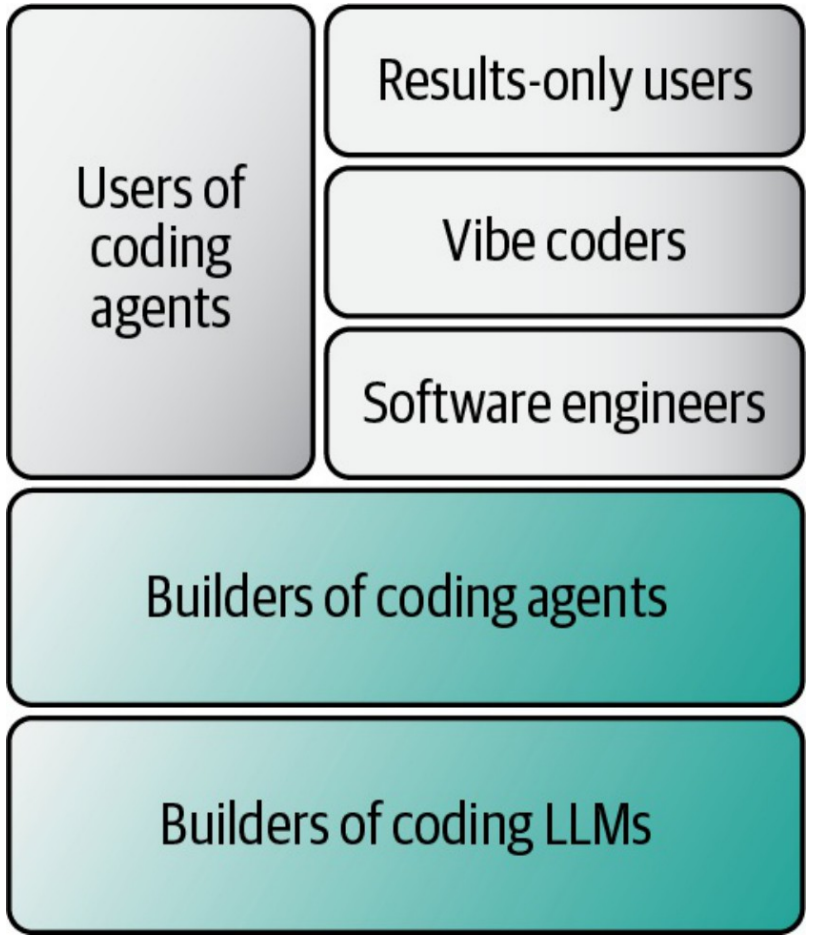
Some want only a chart or analysis and may never see the code. Vibe coders build software through natural language. Software engineers read, review, and direct the agent.

Two kinds of builders

One group builds the agents, assembling the tools, context, and loop around a model. The other trains the coding LLMs that everything depends on.

Different needs

The people who use and build code agents have very different needs. The figure lays out the cast, from end users to professional developers and AI researchers.



Grootendorst & Alammar (2026). Fig. 10-1, PDF p. 502

Three kinds of users, two kinds of builders

Non-code output
These users want a chart or an analysis. They may never see the code the agent writes and run.

Vibe coders
They want working software and build it through natural language, letting the agent handle the details.

Software engineers
They read, review, and direct the agent inside a codebase, so they stay close to the code.

Builders
Some assemble the tools, context, and loop around a model. Others train the coding LLMs underneath.

From generating code to running it

Writing code

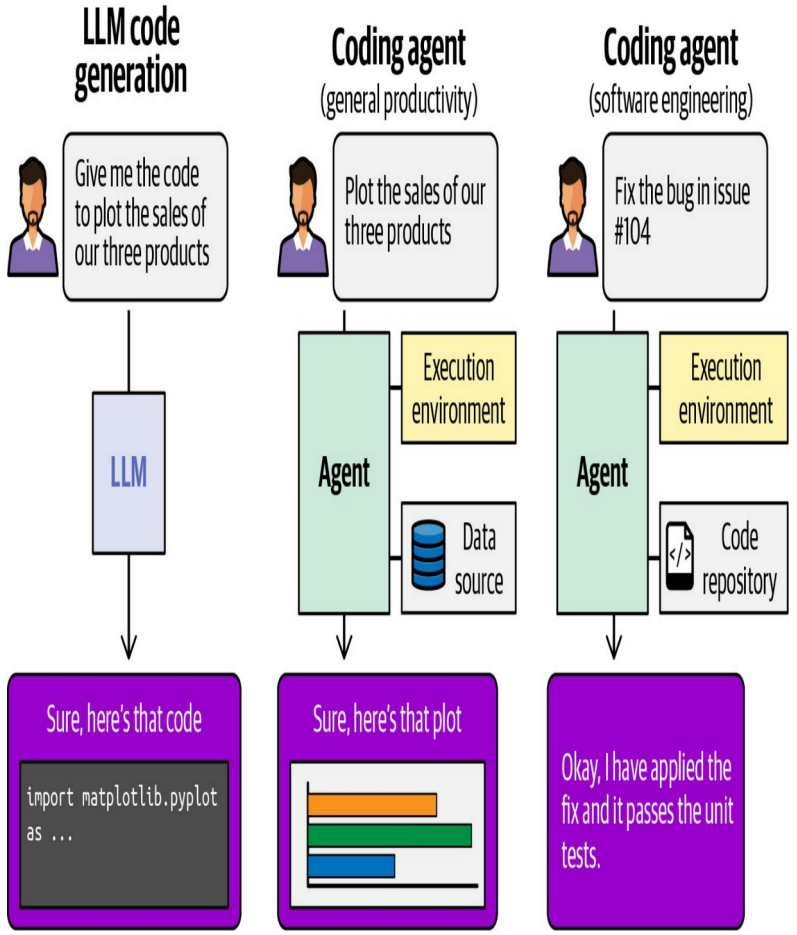
The first example is the familiar one: ask an LLM, often in a playground, to write a piece of code. Nothing is executed.

Run and present

In the second, the LLM writes code, runs it in an execution environment, and presents the result. The user need not be a developer.

Larger tasks

In the third, the agent helps with software development, solving tasks of tens or hundreds of steps. It relies on a richer execution environment.



Grootendorst & Alammar (2026). Fig. 10-2, PDF p. 503

Code agents for non-coders

A larger group of users

Cursor-like software engineering agents come to mind first, but the chapter starts with users who are not programmers because that group is much larger.

New powers

Code agents give non-coders access to powerful code tools they did not have before. Data analysis is a great example.

A plotting request

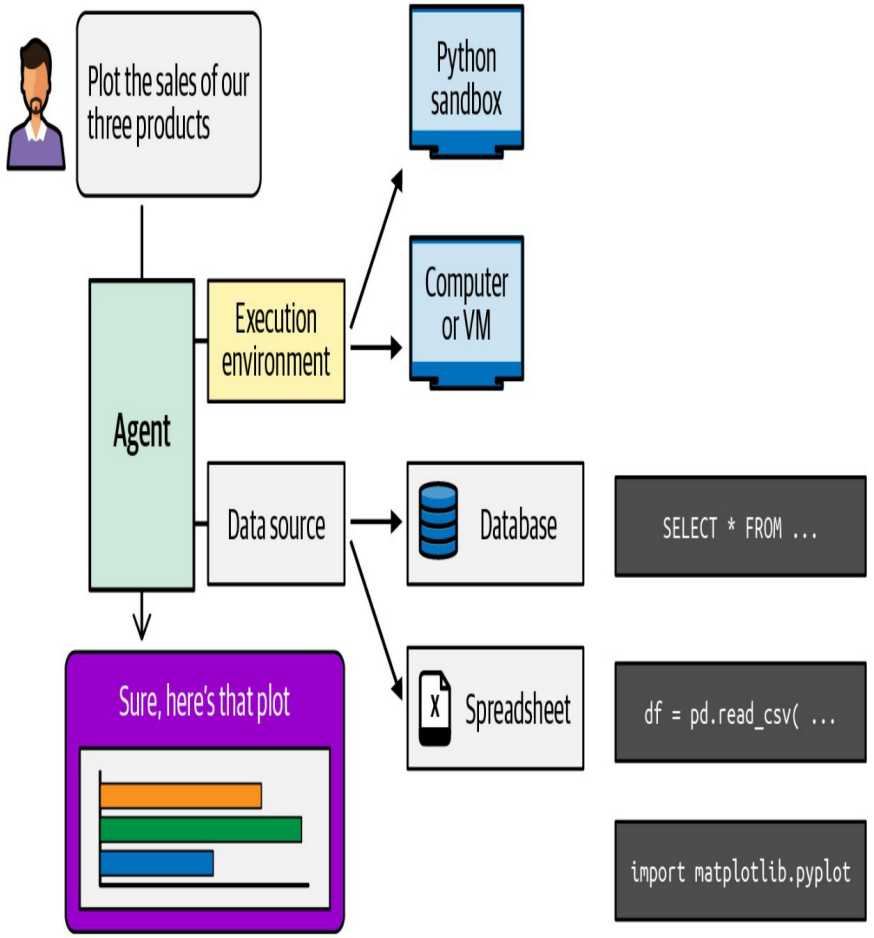
The running example is a request to make a certain plot. The agent must get the data, write code to draw the chart, and handle whatever goes wrong along the way.

An agent that plots by writing code

The request
The user asks for a plot. The agent writes code to retrieve the data and then uses libraries to render the visual.

Data first
Deciding how to get the data comes before drawing anything. The agent chooses a tool, which may itself require generated code.

Hidden code
The user does not necessarily see or interact with any of the code. That is a major lever for the number of people the agent can help.



Grootendorst & Alammar (2026). Fig. 10-3, PDF p. 505

Three ways the agent gets its data

You hand over the file

The simple tool is a Python sandbox that receives the exact file, for example a spreadsheet. The agent writes code with data libraries such as pandas or polars to read and manipulate it.

The agent finds the file

A command-line tool on a virtual machine with a filesystem. The generated code uses command-line tools to search and navigate the filesystem and to retrieve the files.

The data is in a database

A SQL tool gives access to one or more databases. The agent writes SQL, sends it to the tool for execution, and uses the result to retrieve and transform the data it needs.

Plan for failure: troubleshooting

Things go wrong

The data may be missing, or the query the model wrote may contain mistakes or rely on limited information. It is important to plan for the failure cases the agent may face.

Errors as observations

Often an error from the execution environment sent back to the LLM lets it recover on the next step. This is the ReAct loop of Chapter 6: the error is just another observation.

Do not rely on that alone

The more specific failure cases you plan for, the better the end-user experience. The model's ability to recover is a safety net, not a design.

Code tools tie the LLM to an environment

Model decides, tool acts

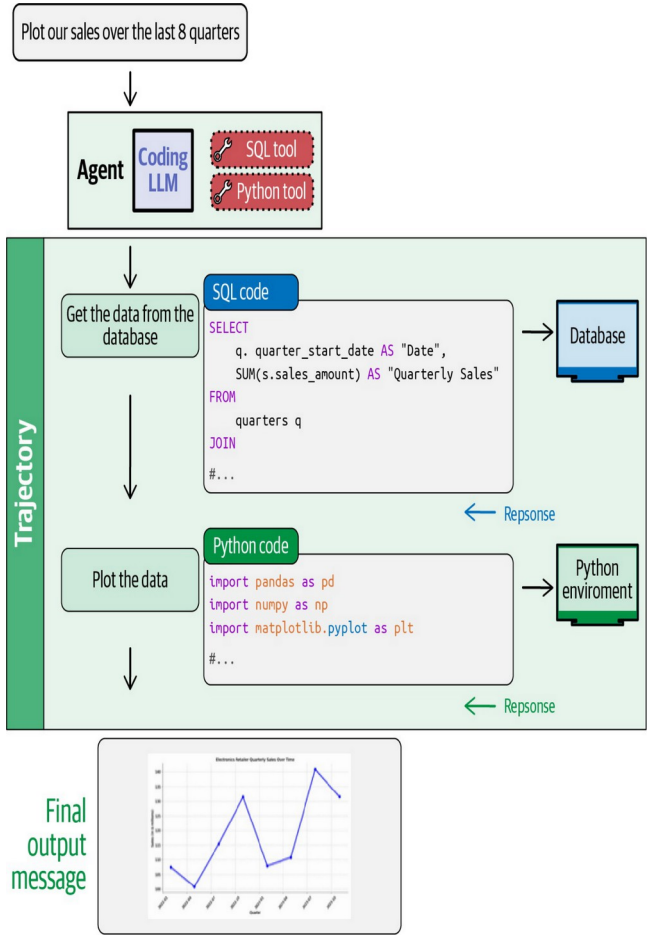
The model decides what it wants to do. The tool carries it out against a real environment: a sandbox, a shell command, or a database.

A two-step trajectory

Another plotting request is resolved in two steps. First a SQL tool fetches the data; then a Python execution tool plots it.

Distinct tools

Each step uses a different tool tied to the appropriate environment, which shows the design space for code agents.



Grootendorst & Alammar (2026). Fig. 10-4, PDF p. 507

File manipulation tools

What they do

Reading, searching, and editing files are among the first things a code agent needs. Distinct tools can each carry their own permissions and checks.

Common tools

Read an entire file (cat); read a section of a file to manage context on long files (sed, head, tail); list a directory (ls); match file names (glob); and search file contents (grep).

Wrappers

These tools are often, but not always, wrappers around command-line tools. They let the agent find files, edit them, create new ones, and navigate a filesystem.

The code interpreter sandbox

Why restrict it

A sandbox is more restricted than a full virtual machine because LLM-generated code is not guaranteed to be safe. An agent could delete files, corrupt a system, or share private data.

Both attackers and accidents

Harm can come from intended attacks, possibly through prompt injection, or from well-meaning but naive agent actions.

Limited and ephemeral

Resources such as memory, processor, and disk are limited. A fresh environment per user or session gives a predictable start and separates users; data is deleted at the end.

Every restriction trades convenience for security

The trade-off

Sandboxes often restrict which commands can run and whether the code has internet access. It is tempting to grant broad access so the agent never gets stuck, but each capability widens what a mistake or attack can do.

Commercial services

Providers offer sandboxes as part of their services, so you may already have used a code interpreter without knowing it. Gemini offers a dedicated code execution tool in its API.

No hosted service needed

The chapter does not require one: the TinyAgent built later runs on a local open model and executes code on your own machine.

Worked example: price cut and profit

The question

A business sells 5,000 units a month at $29, each costing $14 to make. If it cuts the price 10% and volume rises 10%, what is the change in monthly profit?

The code the model wrote

The model wrote code that computes the original profit, then the new price and volume, then the new profit, and finally their difference. The execution tool ran it and returned the numbers.

The result

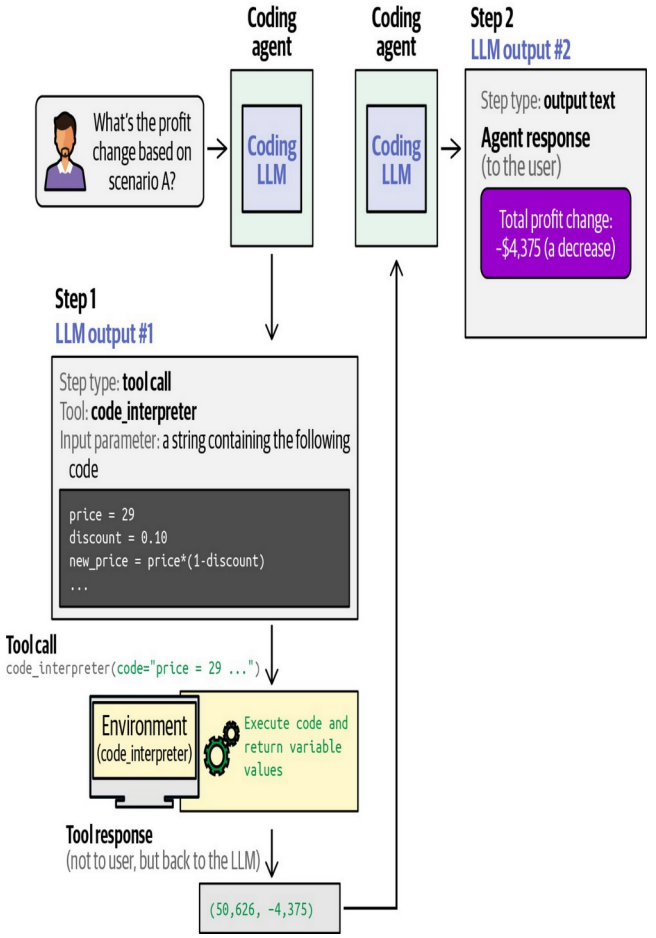
Original profit 75,000; new profit 66,550; change −8,450. The model then answered in prose that profit would decrease by $8,450. The model did the arithmetic by running code, not by guessing.

The agent–interpreter loop

Write and send
The LLM writes the code that calculates the answer and sends it to the tool for execution.

Run and return
The interpreter runs it and the results return to the model, which uses them to decide the next step.

Present
In the example the next step is presenting the final output to the user. Each trajectory costs tokens plus a container.



Grootendorst & Alammar (2026). Fig. 10-5, PDF p. 511

Hosted sandboxes or your own

Same model, other options

You can keep using the same model and use a hosted sandbox from another provider. Examples are Modal, Daytona, E2B, and Together Code Sandbox.

What they offer

These services provide an SDK or API that receives the code your agent wants to run and returns the result, hiding the complexity of managing environments.

Roll your own

With open-source tools such as Open Interpreter or the E2B repository you can configure Python sandboxes in an isolated container you control. This fits specific security or networking requirements.

Command-line tools: more power, more risk

Beyond the interpreter

The next step up gives the model access to the command line of a computer or dedicated virtual machine, so it can inspect or change the environment, for example installing packages.

Who needs it

Software engineering agents such as Cursor or Claude Code tend to require some command-line access to configure environments, run Python scripts and unit tests, and more.

Unfettered access

Giving an LLM unfettered command-line access can quickly become a security nightmare. Recent research treats the execution environment as an adversarial surface, not a trusted backend.

What the security research shows

RedCode (2024)
A benchmark of code prompts that probe agents for dangerous behaviours such as file deletion, privilege escalation, or network misuse.

SandboxEval (2025)
Outlines security-relevant properties of test environments: exposing system, directory, and metadata information; manipulating filesystem structure, contents, and privileges; and starting external communication or dangerous operations.

OpenClaw (2026)
A real-world safety evaluation of a personal agent. Poisoning one dimension of its persistent state (capabilities, identity, or knowledge) raised attack success from about 25% to the 64–74% range. The authors call this inherent in the architecture.

Least privilege is a design-time choice

Approval prompts

Because of these threats, agents in IDEs ask the user to approve each command. The developer is the final decision-maker, permitting actions case by case.

Not enough for everyone

That works when the approver can judge each command, as a developer usually can. It offers little real protection to non-technical users, who cannot evaluate what a command will do.

The builder's job

The safeguard mostly falls to the agent builder at design time: grant only the permissions and data access the task requires, and scope tasks so a wrong action stays contained.

Computer use tools

Like a command line, with a GUI

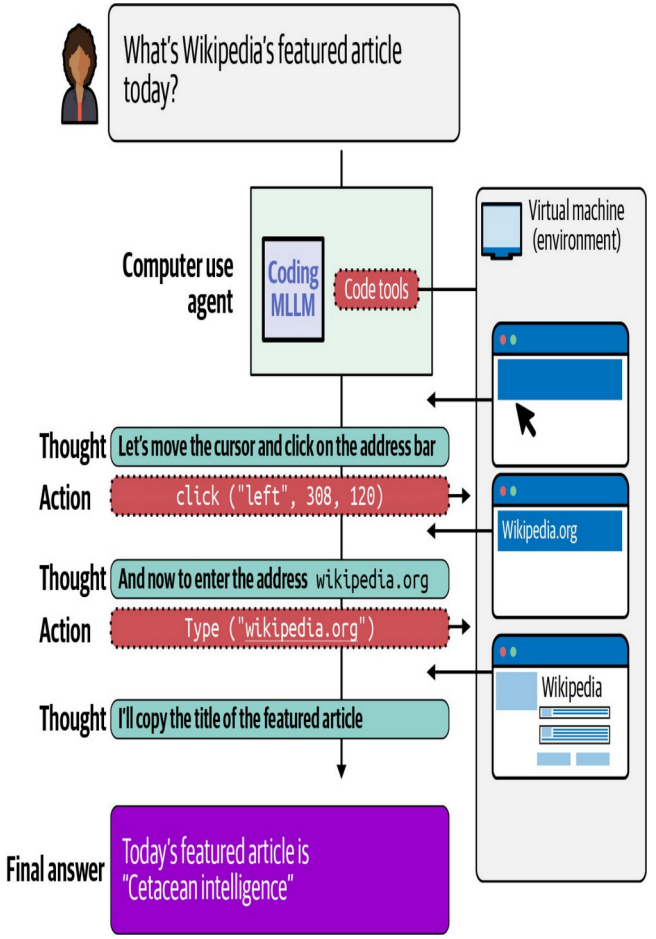
Computer use tools operate on a virtual machine but interact with the operating system's UI. Screenshots feed a vision-language model.

When to use them

They help when the agent must operate applications such as Microsoft Excel, or verify how a UI renders in a specific browser at a specific window size.

The loop

The figure shows an agent working in a virtual machine's browser: a thought, then an action like a click or typing, repeated until a final answer.



Grootendorst & Alammar (2026). Fig. 10-6, PDF p. 514

Code search beyond grep

Structure-aware: ast-grep

Advanced agents benefit from stronger code retrieval. ast-grep works on the abstract syntax tree, so it matches code structure and ignores formatting, variable names, and whitespace.

Semantic code search

A code-embedding model produces dense embeddings. The agent can ask, for example, for a snippet that iterates over a list and get back the most relevant candidates.

Measuring it

CoIR, the Code Information Retrieval Benchmark, measures code retrieval and maintains a leaderboard comparing models.

SQL agents inspect, then query

Data lives in databases

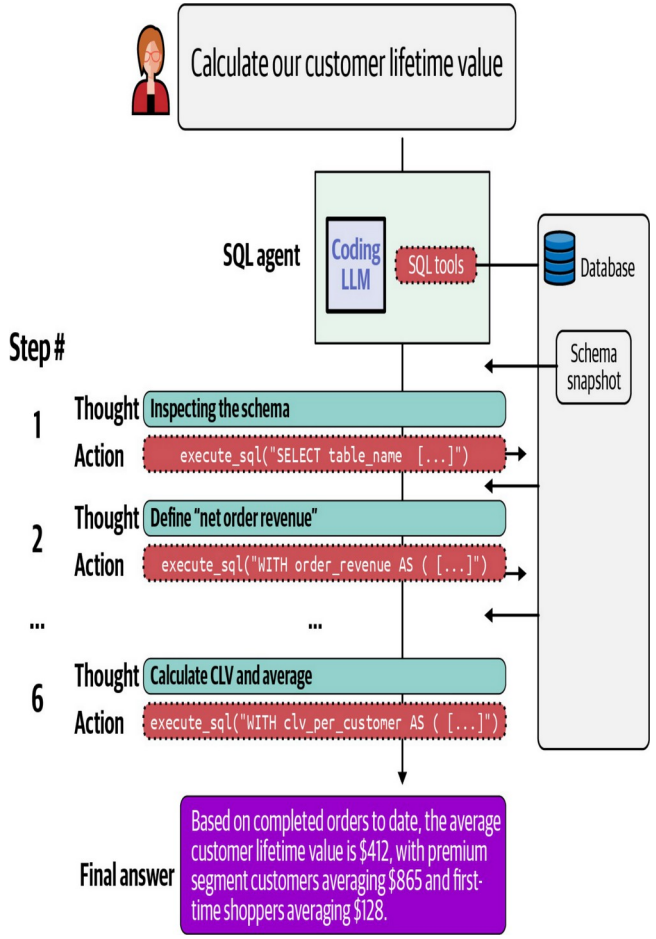
An organization's most important data is often in a database. SQL is code that LLMs can write, for one database or several.

The simplest tool

The simplest SQL tool takes a query and runs it on a pre-configured database. Complex questions may need more than one query.

A multi-step trajectory

In the figure the agent first inspects the schema, then queries tables one after another, and finally answers with a customer lifetime value.



Grootendorst & Alammar (2026). Fig. 10-7, PDF p. 516

Giving the SQL tool the right context

Schema linking

Managing the context of a SQL tool is its own research area, rooted in schema linking from database research. Simpler agents get a general description of tables and columns.

A semantic layer

A more advanced layer identifies relevant columns, may index some of the data in the tables, and captures the language users use, which may differ from what the database stores.

The outerwear example

A user asks a fashion shop's SQL agent for all outerwear products, but the categories are jackets, blazers, and coats. Newer work learns this tribal knowledge from the agent's own query mistakes.

Why code agents need context management

Two constraints

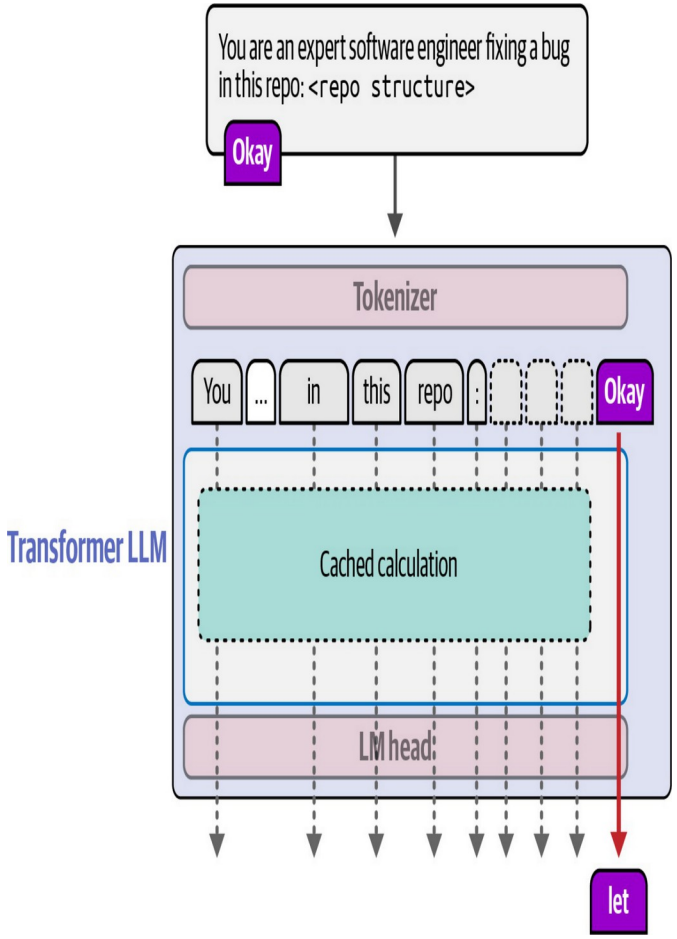
As environments grow, builders must work around the LLM context window and around the latency and cost of very long contexts.

Much more than the request

Beyond the system prompt, persona, and tool descriptions, the context often includes security behaviours and a description of the repository.

Static and repeated

Much of this context repeats on every call. Caching lets the model reuse earlier computation of a fixed prefix instead of recomputing it.



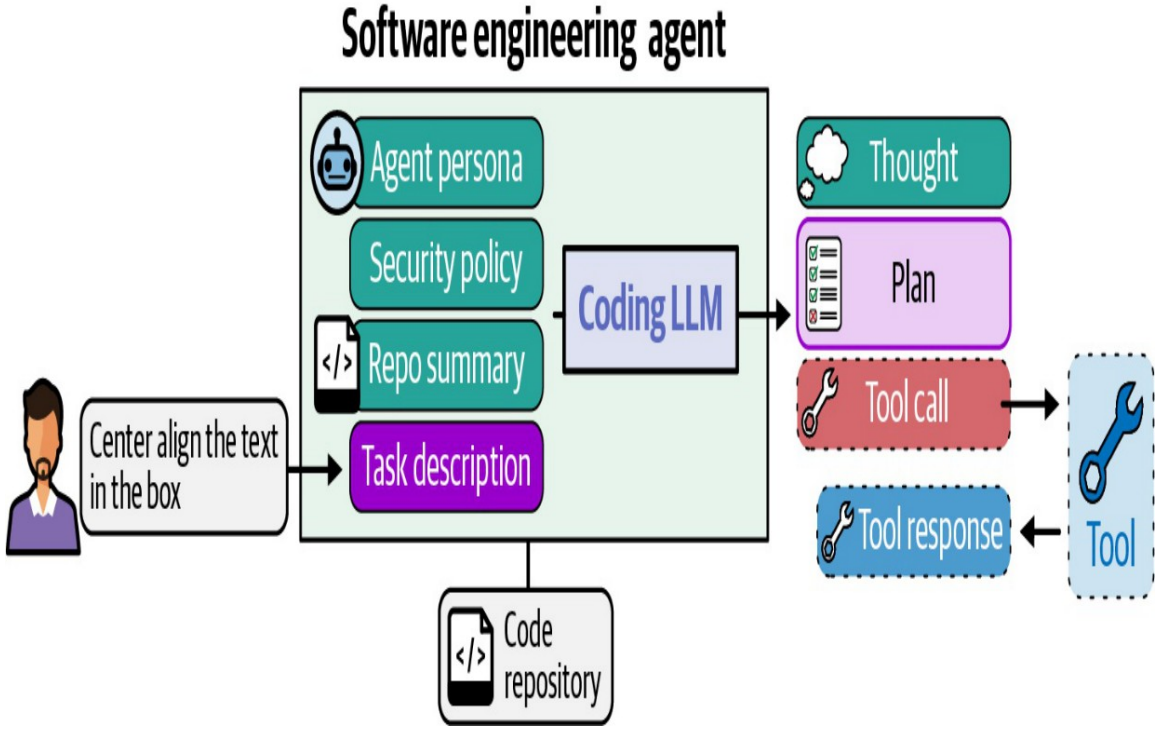
Grootendorst & Alammar (2026). Fig. 10-8, PDF p. 518

What fills the context of a software agent

Tens of thousands of tokens
Even the first call to a software engineering agent can run to tens of thousands of tokens.

Repeated every step
Parts of that context repeat in every later call as the agent works across multiple, sometimes dozens, of steps.

Built from sections
The figure shows the assembled input: agent persona, security policy, repository summary, and the task description.



Grootendorst & Alammar (2026). Fig. 10-9, PDF p. 519

The cache grows with the trajectory

Reuse, do not recompute

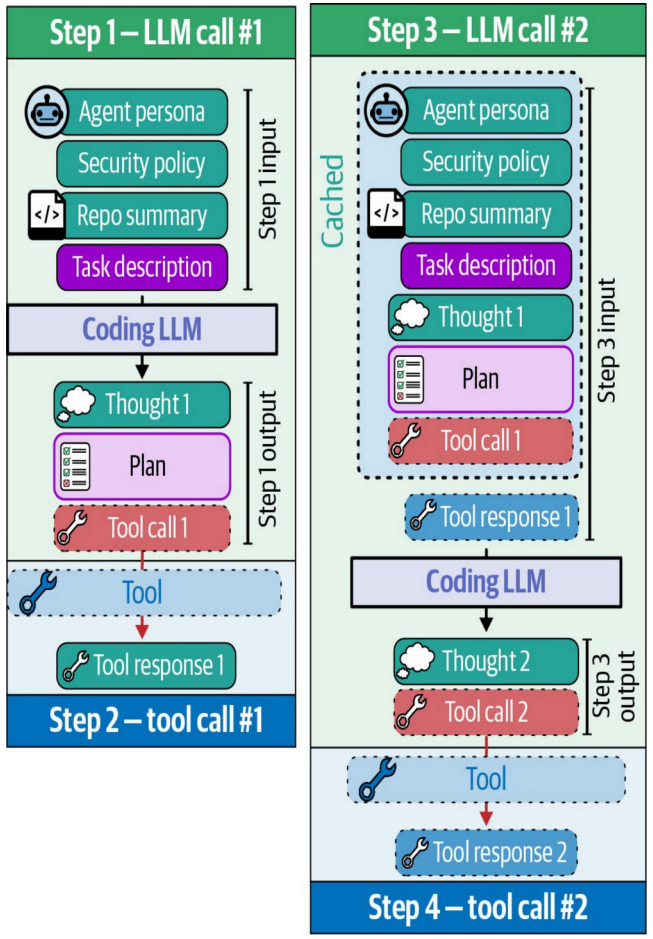
On the next LLM call, instead of processing all input again and paying for it in latency and tokens, the model reuses the first step's cached calculations.

Append only new parts

Only the new parts of the trajectory, the tool response and what follows, are added and computed.

Steps accumulate

Each action and observation is appended to the prompt, so the cached region grows across the multi-turn loop.

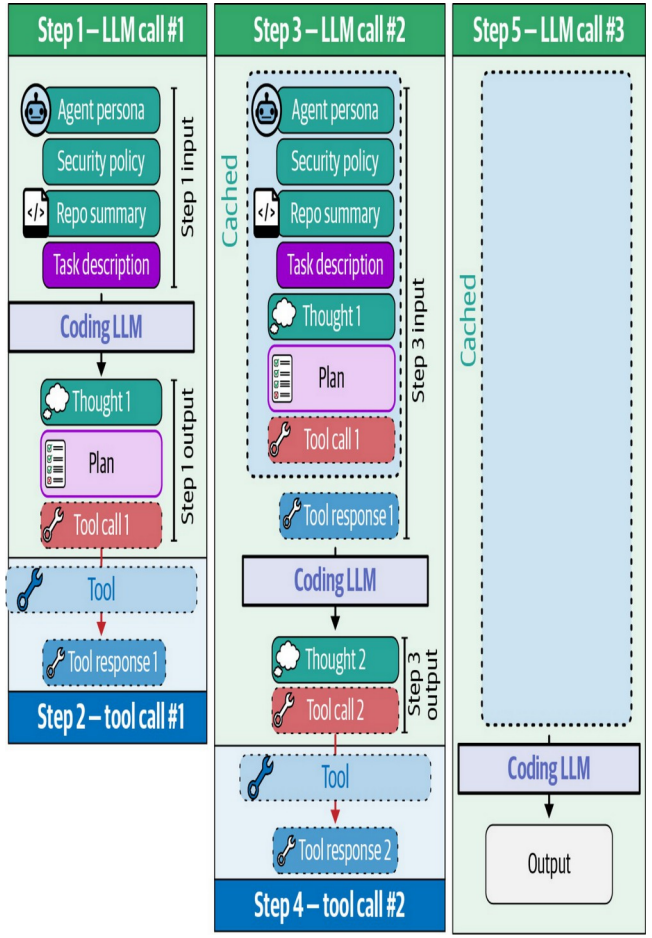


Two rules for a working cache

Identical prefix
The cached input must remain identical. With existing methods, not even a single token in it can change.

Append at the end
All new input must be appended at the end of the cached input, never inserted earlier.

Cache it all
By the third LLM call the entire context accumulated over earlier steps is served from cache as one block. Letting the cache cover the whole trajectory maximizes hits.



Grootendorst & Alammar (2026). Fig. 10-11, PDF p. 522

The repository map

Structure of the repo

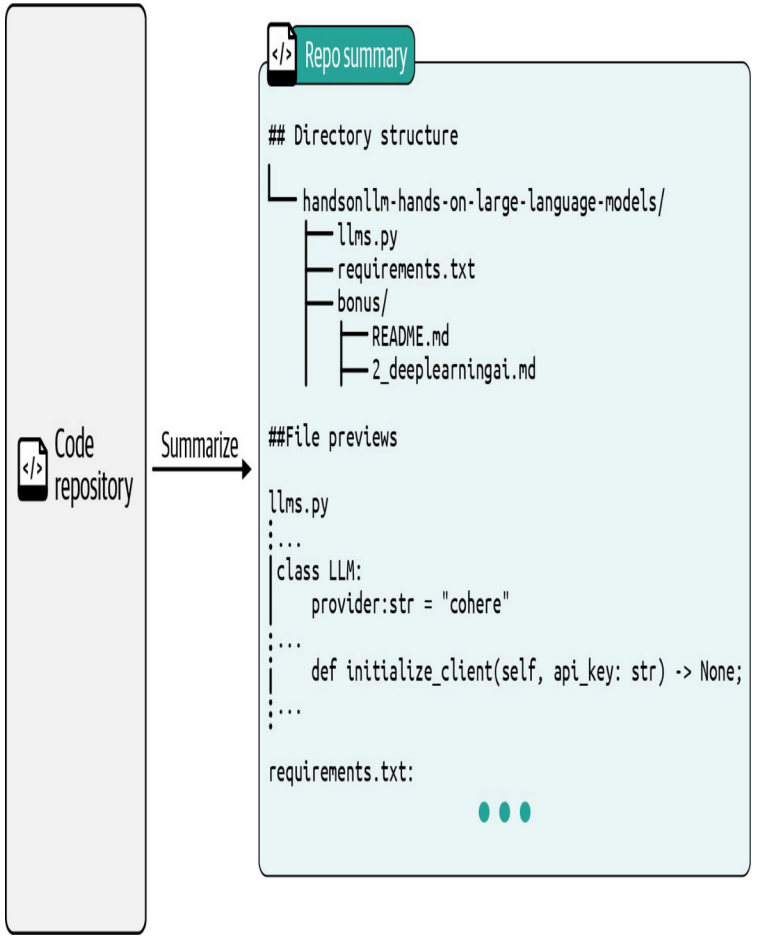
A repository map gives the file and directory structure at some resolution, plus extra information about all or a selection of relevant files.

Gitingest

Gitingest takes a GitHub URL and generates a map with the directory list and the contents of short files.

Aider's style

Aider, an early command-line code agent and precursor of tools like Claude Code, summarizes files with signatures; the figure shows files and previews of key ones.



Grootendorst & Alammar (2026). Fig. 10-12, PDF p. 524

Three ways to build a repository summary

Gitingest

A directory list plus the contents of short files, generated from a repository URL. Simple and quick.

Aider: AST-based ranking

Uses the abstract syntax tree to find important parts such as class and function signatures and uses them as file summaries. A follow-up step ranks and filters to the definitions used most elsewhere in the repository.

CodeMonkeys: LLM summaries

When extra LLM calls are affordable, an LLM reads every file to find the relevant ones. Because the system already samples many candidate solutions, this one-time cost is amortized and is only about 15% of the budget.

Context compaction: static, stable, dynamic

Domain-specific summaries

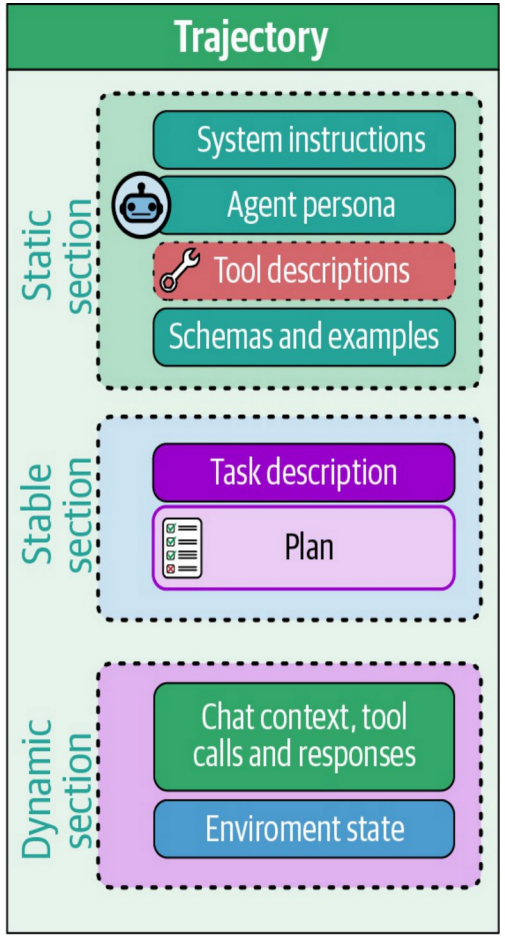
Summarizing a long conversation shrinks context. For code agents the summary should keep environment details, style preferences, when to run tests, and when to commit.

Three sections

The static section stays put, the stable section shifts now and then, and only the dynamic section turns over step to step.

After compaction

The dynamic section shrinks while everything above it stays cached for more and more steps.



Grootendorst & Alammar (2026). Fig. 10-13, PDF p. 526

Code agents for software engineers

A different nature

When the user is a software engineer, code agents take on a different nature. For most developers the first encounter is inside an IDE or code editor.

From autocomplete to agents

These tools started with autocompletion, useful for documentation or a first draft of a function from its signature. Then agents began editing code in several places to solve harder problems.

Less scaffolding over time

Early agents needed close guidance. SWE-agent introduced an Agent-Computer Interface, a constrained set of commands. Newer agents rely less on this because models are now trained on tool use and software engineering.

From functions to whole repositories

A different skill set

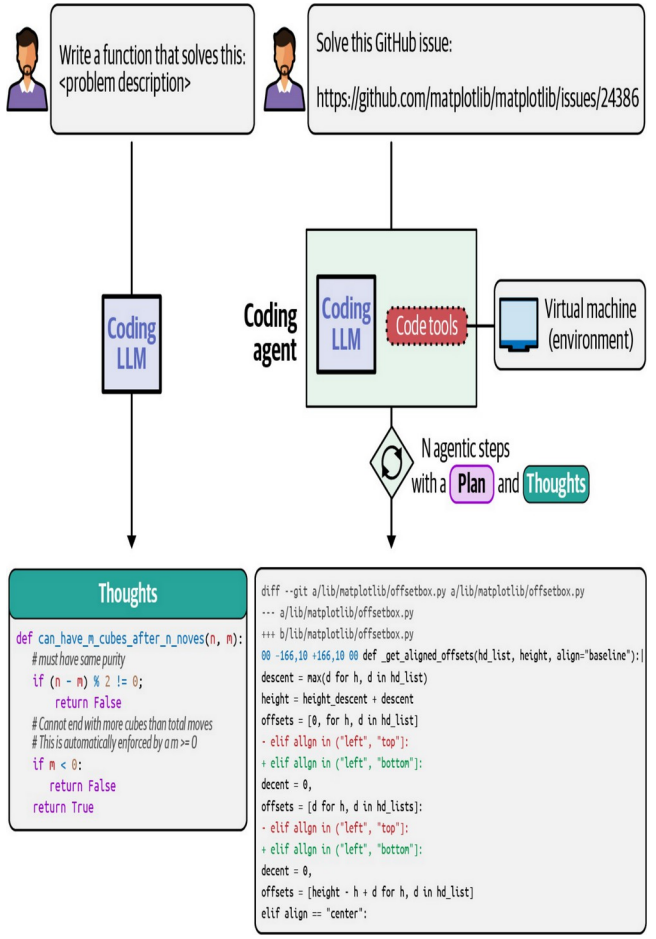
Writing function-level code differs from creating or changing an entire repository, even though both are writing code in the end.

Engineering skills

The agent needs environment management, unit testing, debugging, and the ability to reproduce bugs and investigate their causes.

Reading enough

That means reading the right places in the repository to understand the intention and flow of the program.



Grootendorst & Alammar (2026). Fig. 10-14, PDF p. 528

SWE-bench: a GitHub issue as the task

What it is

The first widely adopted code agent benchmark: hundreds of software engineering issues from open-source repositories, as reported by real users.

How a task works

The agent gets one GitHub issue and a snapshot of the repository at a given time. It must resolve the issue so that a set of test suites passes.

Why it caught on

Issues come from real projects and each ships the repository's own tests, so grading is objective. Early systems solved only a small fraction, leaving headroom, and labs competed on the number.

Planning in software engineering agents

Plans as UI elements

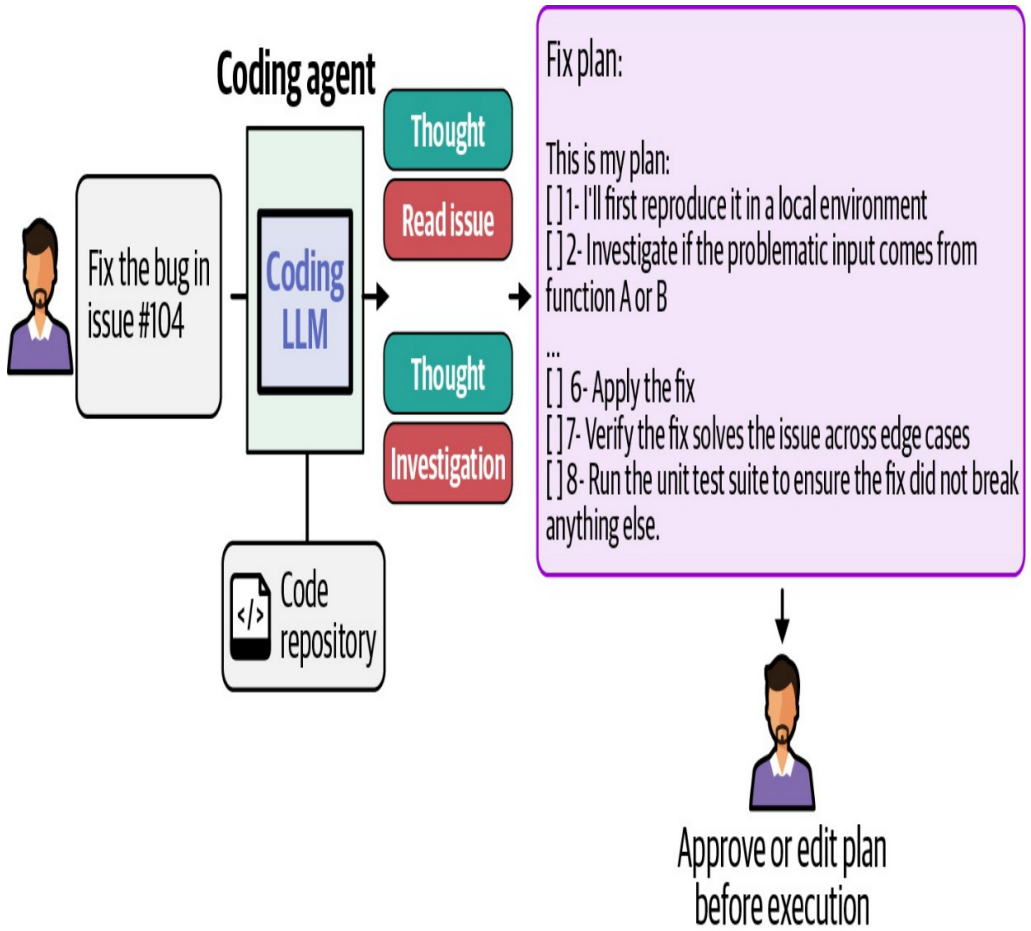
Because tasks are large and multi-step, planning became a key component. The agent writes a plan, returns to it, and updates its status or the whole plan.

A reviewable plan

Code agent UIs show plans and to-do items so the user can track them. The figure shows an agent analyzing a bug report to produce a reviewable modification plan.

Plan, then code

The figure shows a plan made first, then the code changes for each step.



Grootendorst & Alammar (2026). Fig. 10-15, PDF p. 530

Designing the planning step

Separate planner?

Writing a plan and executing it seem like two tasks for two prompts, or even two agents. Some systems assign planning to a dedicated planning agent and steps to dedicated code agents.

Give it the best model

Even in a single-agent system the plan is one of the most important components. It is often wise to assign it to the best available model and let it think for a reasonable time.

Do not assume it is clear

Software tasks often need exploration, research, or asking the user for clarification. The plan should include investigation steps and stay flexible to what they find.

When a workflow beats an agent

Static workflow

A static workflow runs a fixed set of steps you lay out in advance instead of letting the model decide at each turn.

Agent is overkill

Plenty of routine tasks are simple enough that reaching for an agent is more than needed.

Keep other tools within reach

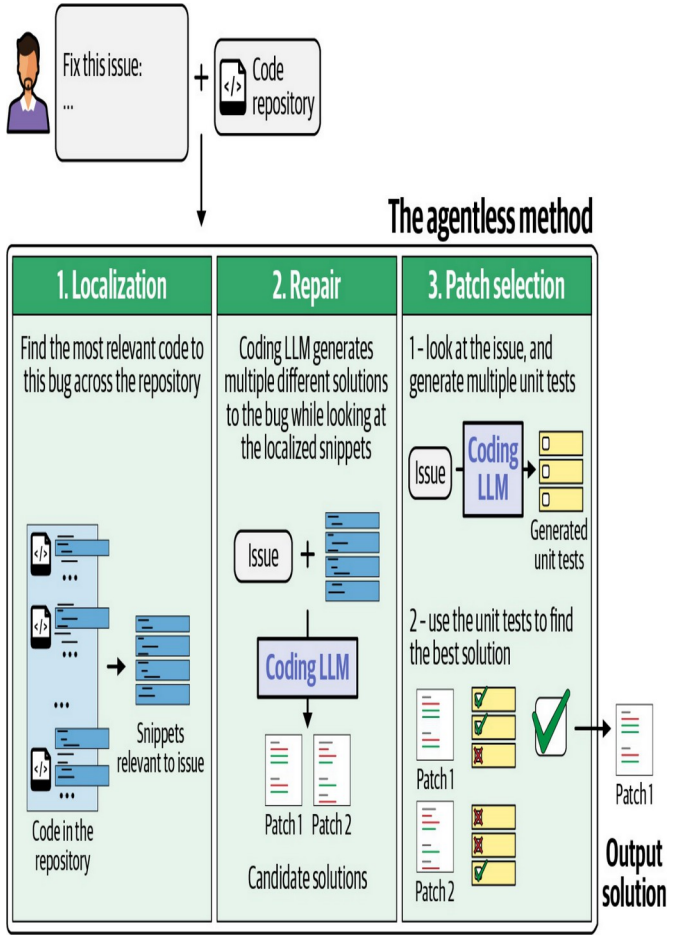
A proficient agent developer picks the tool that fits the problem and does not shoehorn an agent into every job.

Agentless: three fixed phases

Localize
Search the repository for code snippets relevant to the issue. This works much like a RAG system.

Repair
Hand the snippets and the issue to a coding LLM, sampling several candidate patches for more chances.

Select
Have an LLM generate unit tests, then pick the best candidate using them.



Grootendorst & Alammar (2026). Fig. 10-16, PDF p. 532

Two ideas for the agent builder's toolkit

Sample many solutions

Use the probabilistic nature of LLMs: sample multiple solutions at temperatures that allow reasonable variety, so there are several attempts at the issue.

Tests as a ranking signal

Generated unit tests choose among candidates. Because the tests are not fully trusted, the winner need not pass all of them, only more than the other candidates.

Ranking, not pass or fail

Using generated tests as a ranking signal and not a pass or fail signal is the second idea. Together they turn the randomness of LLMs into an advantage.

TinyAgent with code tools

Two tool families

Arguably the most common code tools are file manipulation tools, which let the agent read, list, and write files, and the code interpreter, which lets it execute code it writes.

Four small tools

The book defines read_file, list_files, write_file, and execute_python. Each returns a string, including error messages when something is missing.

Guardrails in the code

execute_python runs the code in a separate process with a 30-second time limit. On failure it returns the exit code with standard output and error, so the model sees what went wrong.

Approval for unsafe tools

Not safe by default

execute_python and write_file are not safe by default. With complete freedom, the agent might corrupt files or take dangerous actions.

requires_approval

Using the parameter from Chapter 5, these tools ask for a yes or no reply before execution so you can inspect whether the action is safe.

Already a coding agent

With the ability to write and run code, TinyAgent is already a basic coding agent. The terminal interface is a better way to use it.

A display that shows the ReAct loop

Four phases

A Display class shows what the agent does: THOUGHT is the reasoning, ACTION the tool it wants, OBSERVATION the tool output, and ANSWER the final result.

Colour coding

ANSI escape codes colour each phase: green for thought, red for action, yellow for observation, and purple for answer, matching the visuals of earlier chapters.

Trying it out

A mock Response with a thought, a tool call to execute_python, and the observation 'Hello World!' shows each step styled clearly in the terminal.

Wiring the display into TinyAgent

Small changes

Integration needs only a display call wherever a behaviour should be tracked: before thinking, after a response, on a tool call, and on an observation.

The same loop

TinyAgent keeps its structure: an autonomy loop of steps, each generating a response, parsing the tool call, stopping on the final answer, or executing the action and storing the observation.

Model and terminal

The book uses Gemma 4 E4B with thinking and native ReAct tool calling, plus a small loop reading user input in the terminal, with an ASCII-art banner.

Running TinyAgent in the terminal

Visible reasoning

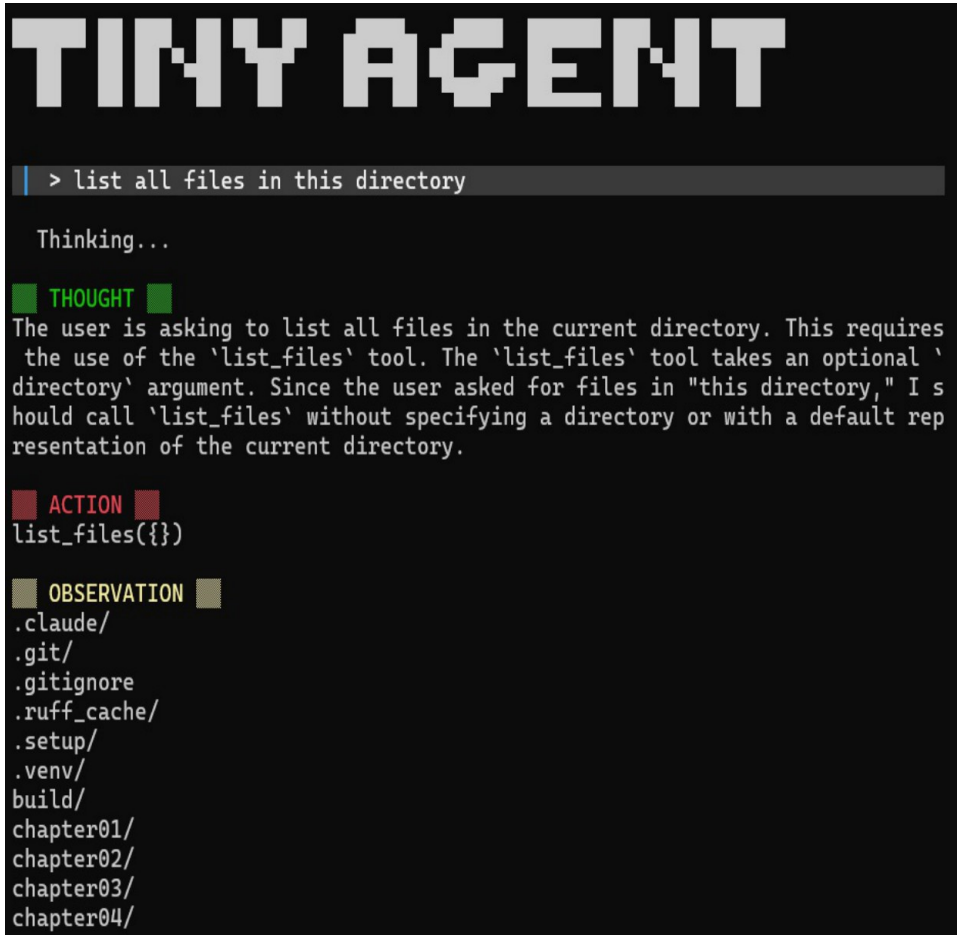
Each task shows THOUGHT, ACTION, and OBSERVATION, which gives an intuition for what happens under the hood.

Try it and compare

See where TinyAgent shines and where it needs work. The model behind it matters a lot, so try a larger one.

A code-trained option

The book suggests North Mini Code: an open Apache 2.0 mixture-of-experts model, 30B total and 3B active parameters, built for agentic coding.



Grootendorst & Alammar (2026). Fig. 10-17, PDF p. 540

What we built, and where the lever is

Updated files

agent.py gains the display. toolbox.py gains the coding tools. Both are updates to what earlier chapters built.

New files

display.py visualizes the ReAct loop, and cli.py runs the agent in the terminal. The rest, llm, memory, planning, tools, evaluator, and trajectory, are unchanged.

The biggest lever

As you experiment you will find the biggest lever on the agent's behaviour is the model behind it. That is where the chapter turns next.

Training a coding LLM

Model limits the agent

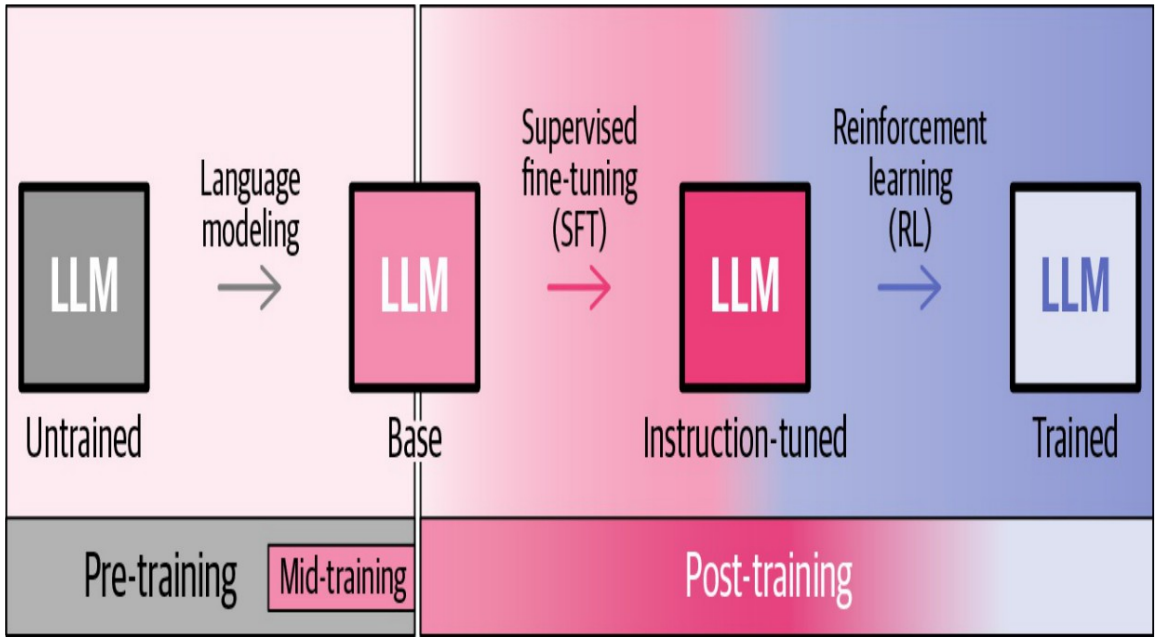
The quality of a coding agent is limited in large part by its model. It must write code well and also be skilled at tool use and software engineering tasks.

Three stages

As the figure shows, training moves through language modeling, supervised fine-tuning, and reinforcement learning.

What differs

Compared with a generalist LLM, the main differences are the data and the role of reinforcement learning.



Grootendorst & Alammar (2026). Fig. 10-18, PDF p. 541

What a code LLM needs: data and RL

Composition

These models see far more code at every stage. Qwen3 Coder, for example, was pre-trained on 7.5 trillion tokens, 70% of them code.

Data that looks like use

Tool-use trajectories, such as running grep and acting on it. Multi-step tasks over several turns. Code paired with unit tests. Repository-level issues paired with the diff that closes them.

RLVR at scale

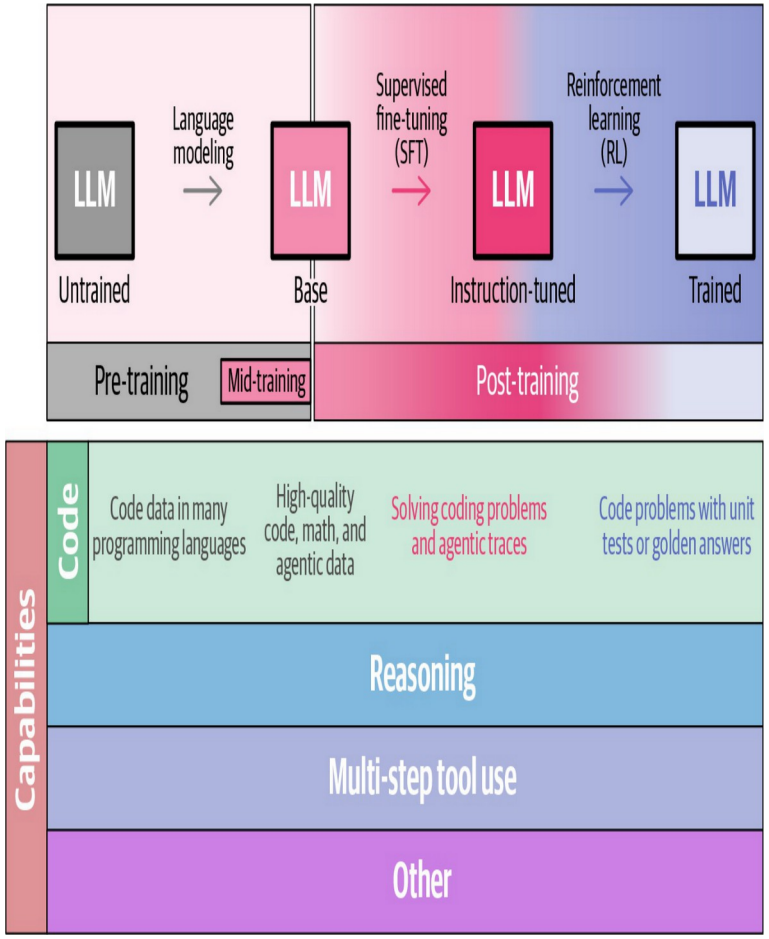
Reinforcement learning is now a major component. Where RLHF polished behaviour, coding LLMs are trained with large-scale reinforcement learning with verifiable rewards, as covered in Chapter 2.

Coding datasets at each training stage

Stage by stage
The figure shows the types of code-related data used in each stage of training a coding agent.

More than code
Reasoning, multi-step tool calling, long context, and planning matter just as much to an LLM that powers a coding agent.

Good practice in data
Trajectories should show proper engineering: how to reproduce issues, how to use coding tools and common libraries.



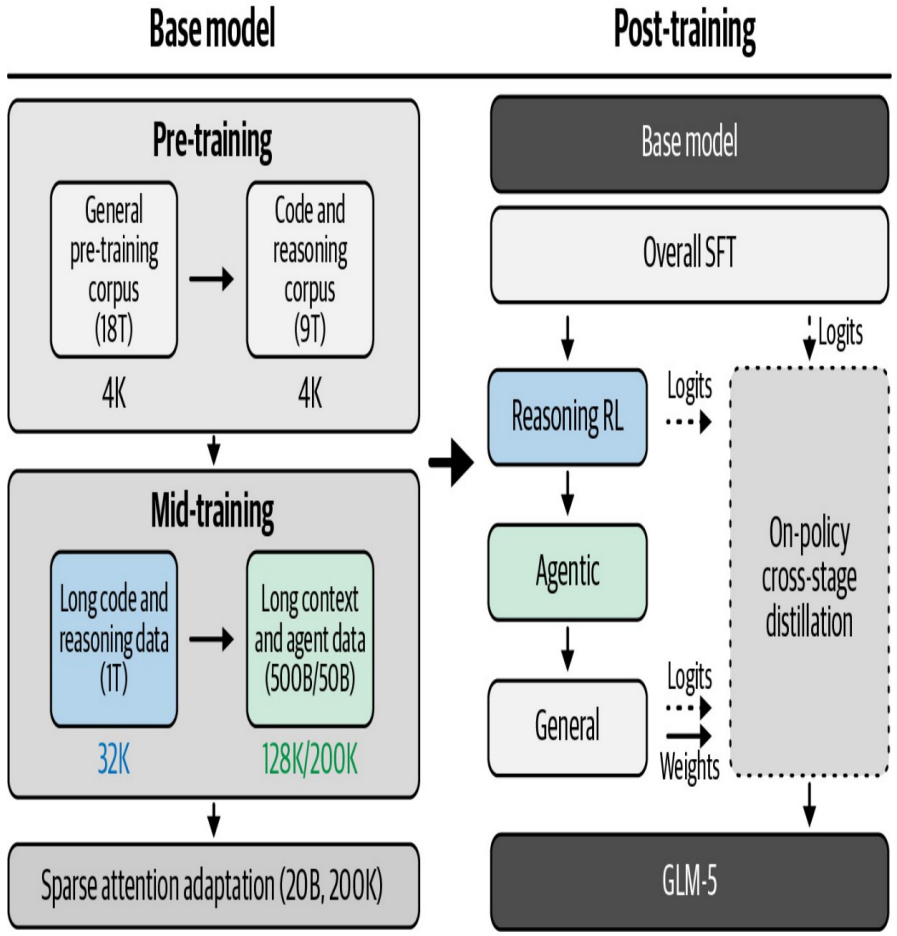
Grootendorst & Alammar (2026). Fig. 10-19, PDF p. 543

How much data: GLM-5's pipeline

Pre-training
On the order of 28 trillion tokens of pre-training.

Mid-training
Context is extended out to 200K with long code and agent data.

Then SFT and RL
SFT, then RL split into reasoning, agentic, and general stages, then a final distillation step.



Grootendorst & Alammar (2026). Fig. 10-20, PDF p. 544

The synthetic data feedback loop

An inflection point

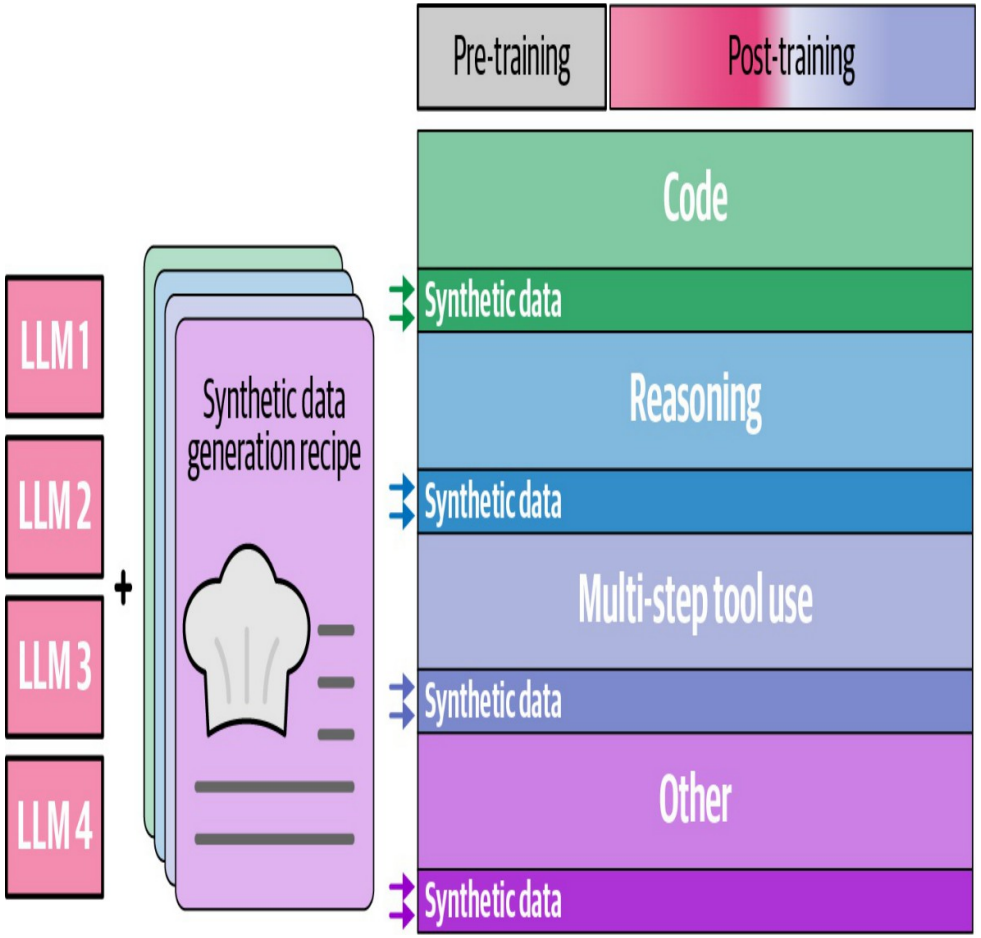
Around 2024 LLMs became a major source of training data for the next generation. Much of Llama 3.1's post-training data was generated with Llama 3.

Models training models

Qwen2.5-Math and Qwen2.5-Coder helped create data for Qwen3 and Qwen3-Coder. DeepSeek-R1-Zero generated many of the reasoning traces for DeepSeek-R1.

A recipe

The figure shows a recipe in which models produce data for pre-training and post-training: code, reasoning, multi-step tool use, and more.



Grootendorst & Alammar (2026). Fig. 10-21, PDF p. 545

Choosing the environment for a task

Plot an uploaded file

A Python sandbox with the file is enough: narrow access, ephemeral, no network. The least privilege that works.

Find files on a machine

A command-line tool on a virtual machine is needed, so the risk is higher. Scope it to the folders the task needs.

Answer from a database

A SQL tool with a read-only account and a schema or semantic layer, so queries cannot change data.

Test a web page layout

A computer use tool with a browser, so the model can see rendering at a given window size.

Agent, workflow, or agentless?

Weekly report from fixed steps
A static workflow. The steps never change, so a model deciding each turn adds cost and risk.

A bug report in a large repo
Either. Agentless fits if the fix is localized; an agent fits if the cause needs exploration.

An open-ended refactor
An agent with a plan that includes exploration steps, tests, and user review.

Many candidate fixes
Sample several, use generated tests to rank them, and let the best pass more tests than the rest.

Teaching addition. Concepts: Ch. 10, PDF pp. 530–533

Governing code agents

Permissions

Decide what the agent may read, write, and run before it starts. Least privilege is a design decision recorded in writing.

Approval and logging

Approvals help only when the approver can judge the action. Keep the trajectory of commands and results for review.

Untrusted inputs

Files, web pages, and tool results can carry instructions. Treat the environment as an adversarial surface.

Cost and data

Containers and tokens cost money. Free-tier prompts may be used by providers, so keep sensitive data out.

Teaching addition. Concepts: Ch. 10. Cross-reference: governance half of the course

Activity: design a safe plotting agent

The scenario

A non-technical manager uploads a spreadsheet of monthly sales and asks the agent for a chart of the last eight months.

The design task

List the tools you would give the agent, what each can access, and which, if any, need approval. Say what the manager sees and does not see.

The failure cases

Name three things that could go wrong, such as a missing column or an empty file, and write how the agent should recover or ask for help.

Learning check with worked answers

1. Why is code a good target?
It can be verified by running or testing it. That gives a clear signal for evaluation and a reward for RLVR training.

2. Why must cached input not change?
The cache reuses computation of an identical prefix. Even one changed token breaks it, so new input is appended at the end.

3. What does agentless rely on?
Localization, sampling several patches, and generated unit tests that rank candidates, with no agent loop.

4. Why more code in the data?
The model must write code and also use tools over multiple steps, so it needs tool-use, test, and repository data too.

Chapter summary

Agents and tools

Code agents extend code generation into multi-step problem solving: write, execute, debug, and iterate. Tools include file tools, sandboxes, command lines, computer use, code search, and SQL, with least privilege to manage risk.

Engineering and TinyAgent

Software engineering agents use caching, repository maps, and compaction. SWE-bench, planning, and agentless frame the task. TinyAgent gained file and execution tools and a terminal display.

The model underneath

Coding LLMs are built with pre-training, SFT, and RL, with far more code, tool-use, and engineering data. RLVR turns verifiability into a training signal, and synthetic data lets models train the next generation.

Afterword: a gift that keeps moving

A gift, not a transaction

The authors say the methods and habits of mind in the book came to them from people who wrote their ideas down and let them go. Drawing on Robin Wall Kimmerer, a transaction closes; a gift does not.

It must keep moving

A gift asks one thing of whoever receives it: that it keeps moving. Held still, hoarded or buried, it stops being a gift.

Build well

The authors hope readers build things that outlast the book, reach people they will never meet, and are good to them. You answer for what your work does there.

Core distinctions and source guide

Conceptual distinctions

A code agent writes code, executes it, reads the result, and iterates. Its tools tie the model to an environment, its context must be managed for cost and cache reuse, and its model is trained with code, tool-use, and verifiable-reward data.

Primary source

Maarten Grootendorst and Jay Alammar. An Illustrated Guide to AI Agents. O'Reilly, first edition, 2026. Core coverage: Chapter 10. Selected connections: Chapters 2, 3, 4, 5, and 6.

Reading the attributions

Figure numbers identify original illustrations. PDF page numbers count the cover as page 1. Source details and further explanation appear in speaker notes. Labeled teaching additions are instructor-created examples and activities.

Source: Supplied 584-page PDF